Machine instruction, and High Level Programming Languages

Before We Start

- Lab Info
  - Week 4 to 7
  - Find you session in e-Bridge
  - Refresh e-Bridge timetable if you don't see it
- In-class quiz
  - account for 30% of you full grade.
  - Time: 19:00-20:00, Friday, Week 10 (**Nov. 20th**)
  - Assesse knowledge from week 1 to week 9
  - Type of questions: MCQ, gap filling, and Assembly Programming.

Overview

- High Level Programming Languages (HLL)
- Compiler & Interpreter & Assembler
- Linking
- Typical Programming Process
- Algorithms

Languages

- Programming languages (also known as Formal languages)
  - High Level Languages (HLL)
  - Assembly
  - Machine Language
- Translator between programming languages
  - Compilers
  - Interpreters
  - Assemblers

Ingredients of Language

- Symbols: a set of words and punctuation (in computing, words and punctuation are collectively known as tokens)
- Grammar (also known as syntax): a set of rules for putting symbols together to get valid statements
- Semantics: a set of rules for interpreting the meaning of a grammatically valid statement

Compare with Nature Language

- Natural languages aren't rigidly governed by grammar rules in everyday use.
  - "I might could get one of them there computers."
- Natural languages can be ambiguous
  - "Fair is foul, and foul is fair."

Programming Languages

- Completely described and rigidly governed by formal rules.
  - Cannot be ambiguous
  - Words and syntax must be exactly correct in every way

High Level Languages (HLL)

- **H**igh **L**evel (Programming) **L**anguages
  - Easy for human to understand.
  - Highly standardized that can be used on mulitple different types of computers.
  - Many differnet ones available, each originally designed and optimized for specific task.
    - C/C++ for operating system
    - Java for Web and Mobile
    - Python for AI
  - Actually used broader than original designs.

HLL Example

- Hello world program in Java

```
System.out.println("Hello, World!");
```

- Hello world program in C

```
printf("Hello, World!");
```

Assembly (not HLL)

- Assembly program sometimes is also called machine insturctions
- Human-readable, but not as friendly as HLL
- Specific to a particular CPU family
  - Intel/AMD x86 (PCs, servers, some handhelds)
  - ARM (handhelds such as smartphones and tablets)
  - IBM POWER (server computers)
  - A program in x86 assembly language cannot be directly run on a machine with an ARM CPU. *

Assembly Example

On intel CPU

```
pushl %ebp
movl %esp, %ebp
subl $8, %esp
subl $12, %esp
pushl $.LC0
call printf
addl $16, %esp
leave
ret
```

On IBM POWER4

```
mflr 0
stw 31,-4(1)
stw 0,8(1)
stwu 1,-64(1)
mr 31,1
lwz 3,LC..1(2)
bl .printf
nop
lwz 1,0(1)
lwz 0,8(1)
mtlr 0
lwz 31,-4(1)
blr
```

Machine Languages

- Machine languages sometimes is also called bindary code
- Not human-readable, except with immense effort
- Binary code that CPU understands directly
- Binary representation of the CPU family's assembly language

Machine Languages

On intel CPU

```
1011110101010001
0101011110101001
1011101010100001
0101101011101000
...
```

On IBM POWER4

```
0111010101000010
1011010111010001
0101010010101010
1101010101011010
...
```

Semantic gap

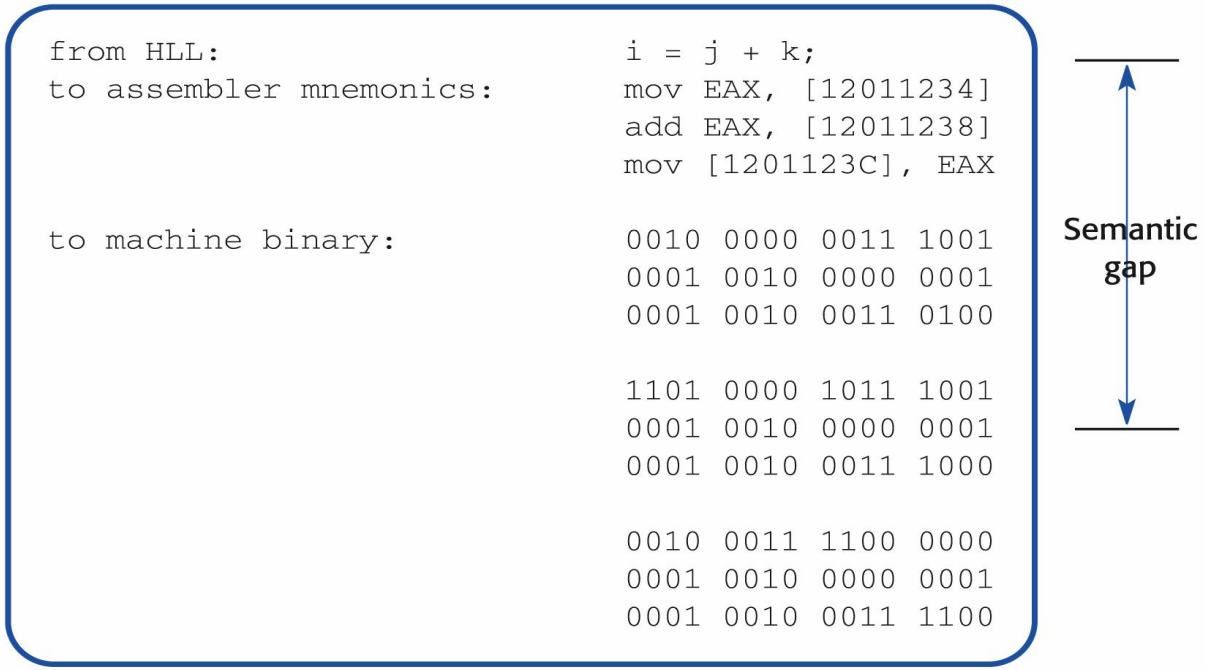


Fig. 2.3 Equivalent HLL, assembler and machine code.

© Pearson Education 2001

- Enormous difference between the way human languages expressing ideas and actions and the way computer instructions representing data processing activities.

Translation: compiler

- Translation is done by special programs such as:
  - **Compilers**, translating HLL instructions into machine code (assembly instruction, or machine bidary code) before the code can be run on the machine.
  - Must be done before executing the program
  - Example compiled languages: C, C++, etc

Translation: Assembler

- Translation is done by special programs such as:
  - **Assemblers,** translating machine instructions (like MOV, ADD, etc) into their binary codes.
  - Like a compiler, but for a low level assembly language
  - You can ask the compiler to translae a program from HLL into assembly code or into bindary code

Translation: Interpreters

- Translation is done by special programs such as:
  - **Interpreters**, translating HLL instructions into machine code on-the-fly (actions immediately performed).
  - Converts and executes one statement at a time.
  - Conversion and execution are interlaced.
  - Example interpreted languages: Java, SAS, etc.
- Python is a special HLL that use compiler and interpreter jointly

Linking

- Big programs usually are divided into several separate parts or **modules.** (Why?)
- Each module has to be designed, coded and compiled.
- There are frequent occasions when code in one module needs to reference data or subroutines in another module.

Linking Flow

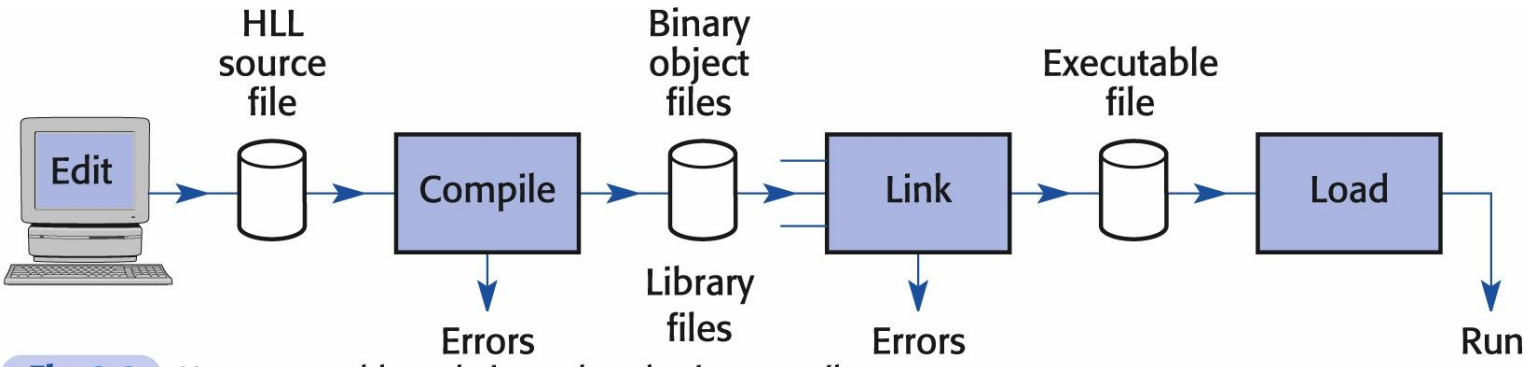


Fig. 2.4 How executable code is produced using compilers.

© Pearson Education 2001

- In practice, assembler and the linker are invoked automatically by the compiler.
- What do we do with compile-time errors?
- What do we do with link-time errors?
- What do we do with load-time errors?

Linking (cont.)

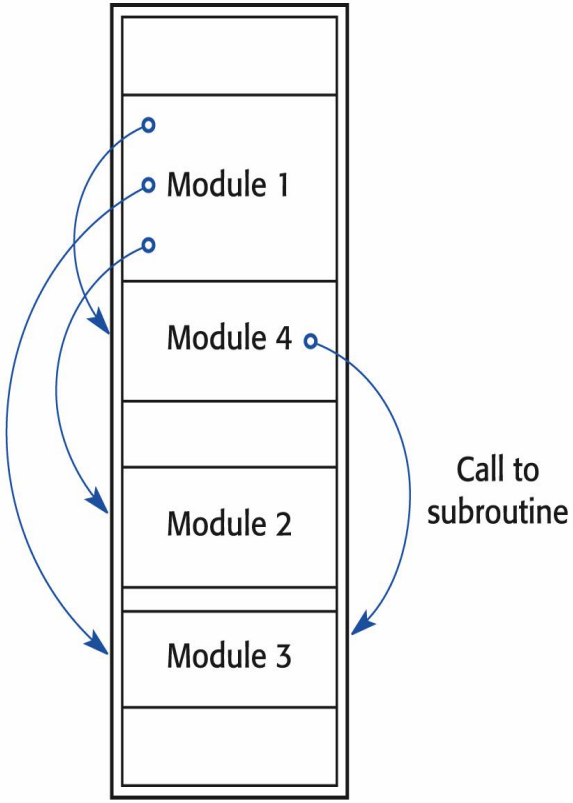


Fig. 2.5 Modules with external references.

© Pearson Education 2001

Linking (cont.)

- A compiler can translate a module into binary codes, but it cannot resolve those references to other modules. (Why?)
- Those *external references* remain symbolic after the compilation, until the linker gets to work.
  - The linker is to join together all the binary parts.
  - The linker will report errors if it cannot find the module or code referred to by those external references.

Output log of a building process by Microsoft Visual C++ 6.0

Deleting intermediate files and output files for project 'voronoi - Win32 Release'.

----------Configuration: voronoi - Win32 Release----------

Compiling resources...

Compiling...

StdAfx.cpp

Compiling...

…

DialogConfirmationUniform.cpp

DialogDistribution.cpp

E:\voronoi\voronoiDoc.h(255) : warning C4244: 'return' : conversion from 'double' to 'float', possible loss of data

…

Generating Code...

Linking...

voronoi.exe - 0 error(s), 21 warning(s)

*

Interpreters vs. Compilers

- Execution of compiled code is much faster than execution of interpreted code. Why?
  - But this disadvantage of interpretation is not a big problem for most applications.
- Interpreters are more suitable for rapid prototyping when a program is frequently modified.
  - Interpreters are more accurate in terms of error reporting. Why?
  - Interpretation can provide uniform execution environment across several diverse computers.

C Program Example

- C language source code → compiler → assembly language → assembler → machine code
- Once we have machine code:
- bindary code → linking and loading → execution

Java Example

- Java source code → compiler (program) → Java "byte codes" → Java interpreter → execution

Interpreters as Virtual Machines

- Interpreters are somewhat similar to the computer hardware (CPU)
  - take one instruction at a time and execute it.

- Because of that sometimes they are referred to as a **virtual machine**
  - Example: JVM, Java Virtual Machine

Code sharing and reuse

- The problem.
  - How to **reuse** existing proven software when developing new systems?
- Three solutions.
  - Source-level subroutines and macro libraries.
  - Pre-translated re-locatable binary object libraries.
  - Dynamic libraries and dynamic linking.

How code can be shared

- A summary diagram.

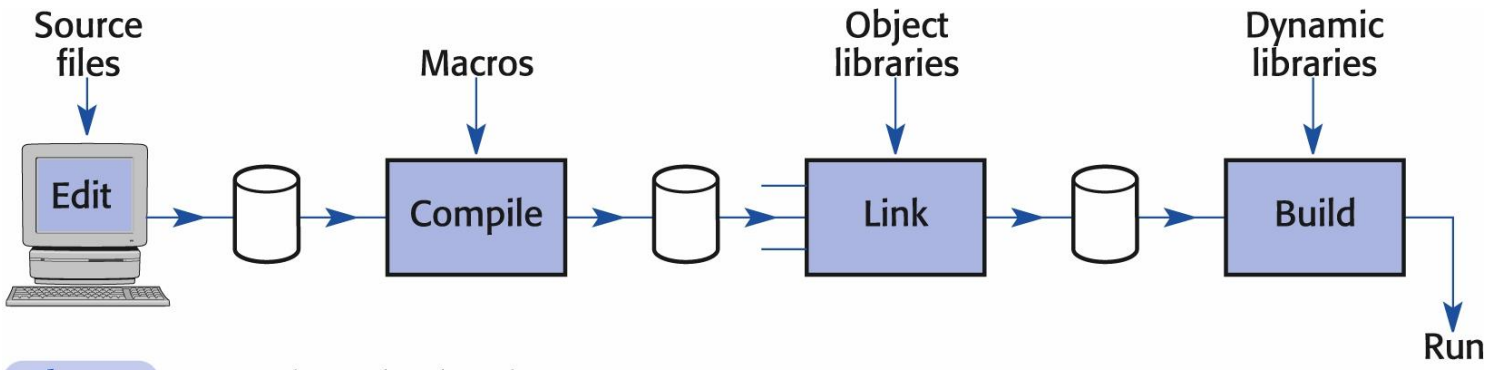


Fig. 2.8 How code can be shared.

© Pearson Education 2001

Macro example

```
macro Print_strings (s1, s2)
printf(“%s”, s1);
printf(“ %s”, s2);
End macro
```

```
main()
{
string a, b;
 …
Print_strings (a, b);
…
Print_strings(“Hello”,”world”);
…
Print_strings(“Game”,”over”);
…
}
```

Expanded macro

```
main()
{
string a, b;
 …
Print_strings (a, b);
…
Print_strings(“Hello”,”world”);
…
Print_strings(“Game”,”over”);
…
}
```

```
main()
{
string a, b;
 …
printf (“%s”, a);
printf ( “%s”, b);
…
printf (“%s”, “Hello”);
printf (“ %s”, ”world”);
…
printf (“%s”, “Game”);
printf (“ %s”, ”over”);
…
}
```

Exercises

- Expand the following macros
  - Print_strings(“Hello my”, “little boy”);
  - Print_strings(“Game is not”,”over yet …”);

Source-level subroutines and macro libraries

- Intention.
  - Take copies of the library routines.
  - Edit them into your new code.
  - Translate the whole together.
- Disadvantages.
  - Who owned the code?
  - Who should maintain it?

Pre-translated relocatable binary libraries

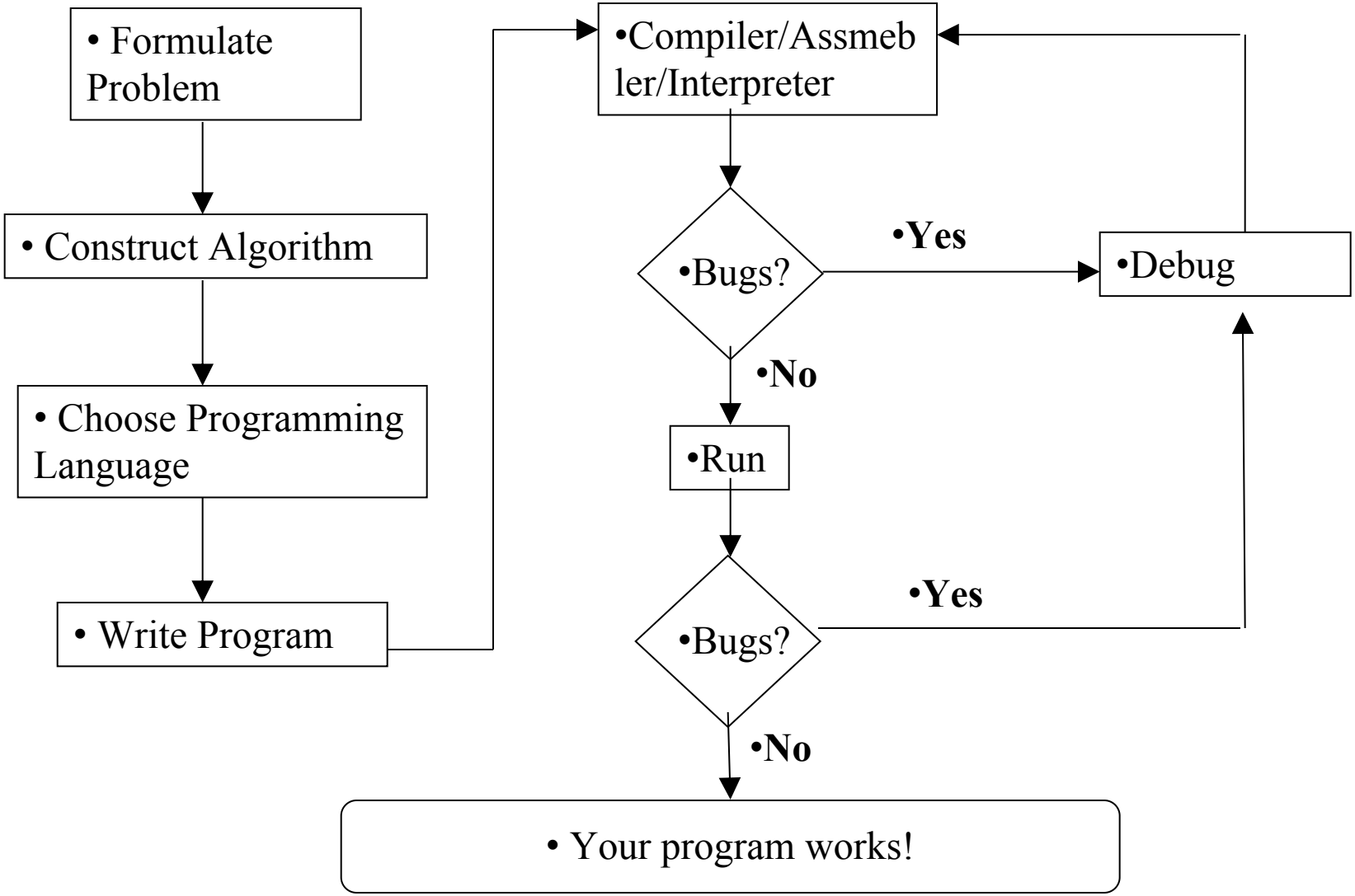
- Intention.
  - Libraries are pre-translated into relocatable binary object code.
  - Can be linked into your new code, **but not altered**.
- Acceptance.
  - Successful, and still essential for all software development undertaken today.
- Disadvantage.
  - Each program is to have a private copy of the subroutines, wasting valuable memory space, and swapping time, in a multitasking system.

Dynamic libraries and dynamic linking

- Intention.
  - Load a program which uses “public” routines already loaded into memory.
  - The memory-resident libraries are mapped, through the memory management system to control access and avoid multiple code copies.
  - Mapped into what?
- Acceptance.
  - Successful through Microsoft’s ActiveX standard.

*

Full Programming Process



About formulating problem

- Example:
  - how many students received “A” in CSI101
- Formulating:
  - List out all students in CSI101
    - Check the score of each student
    - If the stduent's score is A
      - Count it

About algorithm

- An algorithm is: a step-by-step method solves a well-formulated problem
  - algorithm can be written in a natural language (for example, English)
  - algorithm can be written in pseudocode (something that sort of looks like a programming language but isn't as precise)
  - algorithm is not written in any spcific programming language

Algorithm (cont.)

- An algorithm solves a well-formulated problem
  - that has a well-defined set of inputs (which may be empty)
  - using finite resources (for example, finite computing time and storage),
  - producing a well-defined set of outputs (which may be empty).

Algorithm (cont.)

- Formulated Problem:
  - List out all students in CSI101
  - For each student, check if s/he scored A
  - Count those scored A

- Algorithm

```
myAlg(students in CSI101)
    count = 0;
    for student_i in all students of CSI101
        if student_i score == A
            count = count +1;
```

Algorithm vs Program

- An algorithm is language-independent
  - it could be expressed in different languages (for example, English and Chinese)
  - it can be implemented in different programming languages (Assembly, C, Java)
- A program is a language-dependent
  - It may or may not implement an algorithm
  - Same algorithm will lead to different programs when implementing in different programming languages

Q&A

- Name 3 examples of HLLs.
- Translation fills in the semantic gap in computer systems. (True of false?)
- Name 3 different ways of translation. Identify the crucial role of translation under each.
- When compile-time errors occur, what do we do?
- What are the purposes of linking?

Q&A

- ‘Loading’ is carried out before ‘linking’ after a program is compiled. (True or false?)
- Program modules can be compiled separately. (True or false?)
- A compiler can translate a module into binary codes, but it cannot resolve those references to other modules. This occurs when …?
- Library files are usable if linked into your program code. (True or false?)

Q&A

- What is the output of program compilation?
- Name a scenario where interpreters are more useful than compilers.
- Interpreters are sometimes called as virtual machine because … ?
- Mention 3 approaches to code sharing.
- Libraries can be linked into your program code, but not altered. (True or false?)

*

Q&A

- Python is the bset programming language that excels at everything.
- An algorithm designed for Java cannot be implemented in C or other programming language.